

Week 14: Bonus: type inference

Functional Programming in Lean

Learning goals

- ▶ Generate constraints from simple terms
- ▶ Decompose an arrow-type equation
- ▶ Reject a cyclic type with an occurs check

Central idea

Unification solves equality constraints on types.
Let-polymorphism adds generalization.

Lean example

```
inductive Ty where  
  | nat  
  | variable (id : Nat)  
  | arrow (input output : Ty)
```

What to notice

- ▶ $\alpha \rightarrow \text{Nat} = \text{Nat} \rightarrow \beta$ yields two smaller equations
- ▶ $\alpha = \alpha \rightarrow \text{Nat}$ fails the occurs check
- ▶ Lean elaboration solves a richer problem than this teaching model

Live coding

1. Trace constraints for identity
2. Complete occurs
3. Decompose an arrow equality

Pause and predict

Why can α not stand for $\alpha \rightarrow \text{Nat}$?

Exercise connection

Open `exercises/Week14.md` after the lecture.

The complete example is `Course/Week14.lean`; the fill-in version is `Templates/Week14.lean`.